

ORCHESTR

Verified AI orchestration across your tools

Multi-App AI Agent Hackathon — Project Documentation

One useful agent. Three external apps. Every action verified. Orchestr is an AI agent that coordinates operational incidents across Discord, GitHub, and Google Calendar. It investigates a situation, creates an execution plan, performs actions across connected applications, and verifies that the intended state was actually achieved.

Core promise: Act across your apps. Verify every action.

1. The Problem

When an incident happens, information and follow-up work are distributed across communication, code, and scheduling tools. A human has to read the incident discussion, investigate recent code changes, communicate findings, and coordinate the next meeting.

Orchestr turns that fragmented workflow into one multi-step agent task. The goal is not another chatbot, but an agent that can safely change the state of multiple real applications.

2. Product Concept

User: @Orchestr handle this production incident

Orchestr follows a focused, deterministic workflow for the hackathon demo: 1. **Investigate Discord:** Read incident context, reports, and relevant conversation from the #incidents channel. 2. **Investigate GitHub:** Search the target repository for recent commits. 3. **Reason & Plan:** Read the diff of the problematic commit to identify the bug. 4. **Act (GitHub):** Create a Pull Request with a code fix. 5. **Act (Calendar):** Schedule an "Incident Review" follow-up meeting in Google Calendar. 6. **Report (Discord):** Post the PR link and meeting confirmation to the #developers channel. 7. **Dashboard:** Show a concise, real-time auditable action and verification timeline via SSE.

3. Technical Architecture

The architecture has been overhauled for the hackathon to utilize Docker, Python, and the Google Agent Development Kit (ADK).

Core Components: * **FastAPI Backend** (backend/main.py): Hosts the Discord Bot (discord.py), manages the PostgreSQL action ledger, and exposes the REST/SSE endpoints for the frontend. * **Agent Sandbox** (sandbox/agent.py): An isolated container running the Google ADK Agent. It pulls incidents from the backend, uses shell/cli tools (git , gh) to investigate and fix

code, and logs progress back to the ledger. * **PostgreSQL Database:** Stores incident states and the immutable action ledger.

4. Frontend & API Contract (For the Dashboard Team)

The frontend is a minimal execution dashboard showing the live timeline of actions.

API Endpoints: * `GET /api/incidents` - Returns a list of all incidents and their current status (investigating, verified, etc). * `GET /api/incidents/{id}` - Returns the full incident details and completed ledger. * `GET /api/incidents/{id}/stream` - (**Server-Sent Events / SSE**). Connect to this endpoint to receive live JSON updates as the agent acts.

Stream Payload Example:

```
json {"event": "step_update", "data": {"step": "github_read", "message": "Investigating commits...", "status": "in_progress"}} {"event": "step_update", "data": {"step": "github_pr", "message": "Created PR #42", "status": "verified"}}
```

5. Two-Minute Demo Strategy

The demo will follow a single, highly-polished "Happy Path" to guarantee success under time pressure.

- **0:00–0:15 — Problem:** Show a Discord production incident in `#incidents` and mention `@Orchestr`.
- **0:15–0:40 — Investigation:** The Frontend Dashboard lights up via SSE. Orchestr reads Discord and investigates GitHub commits.
- **0:40–1:10 — Execution:** Orchestr clones the repo, fixes the bug, and pushes a Pull Request.
- **1:10–1:30 — Coordination:** Orchestr posts the PR response in `#developers` and schedules the follow-up in Google Calendar.
- **1:30–2:00 — Closing:** Show the Dashboard displaying all green checks. Deliver the product promise.

The key visual: DISCORD → GITHUB → DISCORD → CALENDAR → 4/4 VERIFIED

6. Hackathon Scope (Revised)

Build: * Discord + GitHub + Calendar integrations. * One polished incident workflow (Trigger -> Fix -> Notify -> Schedule). * Live execution dashboard (via SSE stream).

Avoid: * Complex autonomous routing or open-ended demos. * "Knowing when not to act" (Scenario B) has been dropped to guarantee a perfect 2-minute execution flow on the main stage.